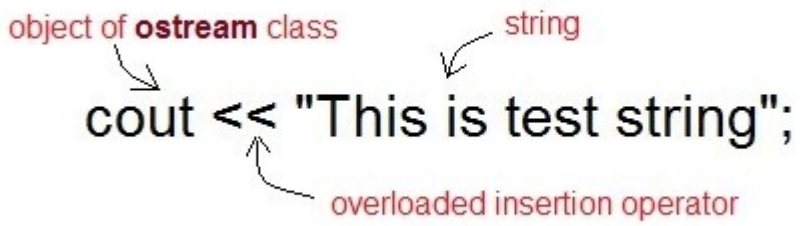
Operator Overloading :

Operator overloading is used to enhance or improvise the functionality of the existing operators , so that they can be applied on user defined datatypes without changing their functionality , priority and associativity of operators.

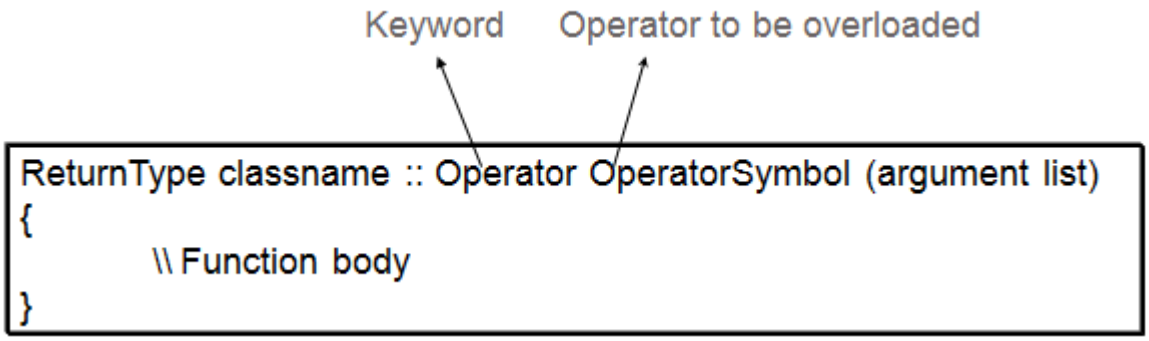
It is a type of polymorphism in which an operator is overloaded to give user defined meaning to it. Overloaded operator is used to perform operation on user-defined data type. For example '+' operator can be overloaded to perform addition on various data types, like for Integer, String(concatenation) etc.



Almost any operator can be overloaded in C++. However there are few operator which can not be overloaded. **Operator that are not overloaded** are follows

- scope operator - `::`
- `sizeof`
- member selector - `.`
- member pointer selector - `*`
- ternary operator - `?:`

Operator overloading Syntax:



Operator Overloading is implemented using :

1. Using Member functions of a class.

2. using Friend functions of a class.

Operator overloading function can be a member function if the Left operand is an Object of that class, but if the Left operand is different, then Operator overloading function must be a friend function.

Restrictions on Operator Overloading in C++

Following are some restrictions to be kept in mind while implementing operator overloading.

- Precedence and Associativity of an operator cannot be changed.
- Arity (numbers of Operands) cannot be changed. Unary operator remains unary, binary remains binary etc.
- No new operators can be created, only existing operators can be overloaded.
- Cannot redefine the meaning of a procedure. You cannot change how integers are added.

Vector India

Operator Overloading Examples

Overloading Arithmetic Operator

In the below example we have overridden the + operator, to add to Time(hh:mm:ss) objects.

```
#include <iostream.h>

using namespace std;

class Time
{
    int h,m,s;
    public:
    Time()
    {
        h=0, m=0; s=0;
    }
    void setTime();
    void show()
    {
        cout<< h<< ":"<< m<< ":"<< s;
    }

    //overloading '+' operator
    Time operator+(time);
};

Time Time::operator+(Time t1)   //operator function
{
    Time t;
    int a,b;
    a = s+t1.s;
    t.s = a%60;
    b = (a/60)+m+t1.m;
    t.m = b%60;
    t.h = (b/60)+h+t1.h;
    t.h = t.h%12;
    return t;
```

Vector India

```
}

void Time::setTime()
{
    cout << "\n Enter the hour(0-11) ";
    cin >> h;
    cout << "\n Enter the minute(0-59) ";
    cin >> m;
    cout << "\n Enter the second(0-59) ";
    cin >> s;
}

void main()
{
    Time t1,t2,t3;

    cout << "\n Enter the first time ";
    t1.setTime();
    cout << "\n Enter the second time ";
    t2.setTime();
    t3 = t1 + t2;       //adding of two time object using '+'
operator
    cout << "\n First time ";
    t1.show();
    cout << "\n Second time ";
    t2.show();
    cout << "\n Sum of times ";
    t3.show();

}
```

While normal addition of two numbers return the sumation result. In the case above we have overloaded the + operator, to perform addition of two `Time` class objects. We add the **seconds**, **minutes** and **hour** values separately to return the new value of time.

In the `setTime()` funtion we are separately asking the user to enter the values for hour, minute and second, and then we are setting those values to the `Time` class object.

Vector India

For inputs, t1 as **01:20:30**(1 hour, 20 minute, 30 seconds) and t2 as **02:15:25**(2 hour, 15 minute, 25 seconds), the output for the above program will be:

1:20:30 2:15:25 3:35:55

First two are values of t1 and t2 and the third is the result of their addition.

Example 2:

example for overloading the == operator in the Time class to directly compare two objects of Time class.

```
class Time

{
    int hr, min, sec;
    public:
    // default constructor
    Time()
    {
        hr=0, min=0; sec=0;
    }

    // parameterised constructor
    Time(int h, int m, int s)
    {
        hr=h, min=m; sec=s;
    }

    //overloading '==' operator
     bool operator==(Time &);
};

/*
    Defining the overloading operator function
    Here we are simply comparing the hour, minute and
    second values of two different Time objects to compare
    their values
*/
bool Time :: operator== (Time &t2)
{
```

Vector India

```
    return ( hr == t2.hr && min == t2.min && sec == t2.sec );
}

void main()
{
    Time t1(3,15,45);
    Time t2(4,15,45);
    if(t1 == t2)
    {
        cout << "Both the time values are equal";
    }
    else
    {
        cout << "Both the time values are not equal";
    }
}
```

```
Both the time values are not equal
```

As the hour value of object `t1` is **3** and for object `t2` it is **4**, hence they are not equal.

Example to Overload the Unary operator:

```
class Time

{
    int hr, min, sec;
    public:
    // default constructor
    Time()
    {
        hr=0, min=0; sec=0;
    }

    // parameterised constructor
    Time(int h, int m, int s)
    {
        hr=h, min=m; sec=s;
    }
```

Vector India

```
     void print()
     {
          cout<<hr<<":"<<min<<":"<<sec<<endl;
     }

    //overloading '++' operator
     void operator++();  // pre increment
     void operator++(int); // post increment
};

/**************** overloading pre increment operator
*************/

void Time :: operator++()

{

     ++hr;

     ++min;

     ++sec;

}


/************ overloading post increment operator
*****************/

void Time :: operator++(int)

{

     Time t;

     t.hr=hr++;

     t.min=t.min++;

     t.sec=t.sec++

     return t;
```

Vector India

```
}

void main()
{
    Time t1(3,15,45);
    Time t2(7,20,35);
    Time t3,t4;

     t3=++t1;

     t4=t2++;

cout<<"t1:";

t1.print();

cout<<"t2:";

t2.print();

cout<<"t3:";

t3.print();

}

output:

t1: 4:16:46

t2: 7:20:35

t3: 4:16:46

t4: 8:21:36
```

Vector India